

**LEMBAR KERJA PRAKTIKUM : ALGORITMA DAN STRUKTUR DATA**

**Analisis Konvolusi Citra Dan Optimasi Strassen**



Oleh :

Nama : Nursyafika

NIM : D121251121

**PRODI TEKNIK INFORMATIKA**

**FAKULTAS TEKNIK**

**UNIVERSITAS HASANUDDIN**

**TAHUN 2026**

## PEMANASAN : SINKRONISASI KONSEP DASAR

### 1. Variabel dan Tipe Data

- Jika memori adalah sebuah gudang besar berisi kotak-kotak, apa perbedaan antara kotak berlabel `int`, `float`, dan `unsigned char`?

Jawaban: Perbedaannya terletak pada ukuran kotak dan jenis barang yang disimpan. Kotak `int` dan `float` berukuran besar (4 byte); `int` digunakan untuk menyimpan angka bulat, sedangkan `float` untuk angka pecahan. Sementara itu, kotak `unsigned char` berukuran sangat kecil (hanya 1 byte) dan khusus menyimpan angka bulat positif kecil antara 0 sampai 255.

- Mengapa untuk data warna piksel (0-255) kita lebih memilih `unsigned char` daripada `int`? Hubungkan dengan efisiensi kapasitas gudang (memori).

Jawaban: Karena nilai warna piksel tidak pernah melebihi 255, sehingga cukup disimpan dalam kotak kecil (`unsigned char`). Menggunakan kotak besar (`int`) untuk barang kecil adalah pemborosan ruang gudang. Dengan `unsigned char`, kita bisa menyimpan 4 piksel di ruang yang sama yang hanya bisa menampung 1 piksel jika menggunakan `int`.

### 2. Array 1D vs 2D (Apartemen Memori)

- Gambar PNG berukuran 100×100 piksel adalah Array 2D. Namun, menyimpannya sebagai satu deretan panjang (1D) di memori. Jika posisi piksel adalah (x,y), jelaskan kenapa rumusnya adalah  $\text{index} = y * \text{width} + x$ .

Jawaban: Bayangkan sebuah apartemen di mana setiap lantai memiliki jumlah kamar yang sama (`width`). Untuk mencapai kamar di lantai `y` posisi `x`, kita harus melewati semua kamar di lantai-lantai bawahnya terlebih dahulu ( $y * \text{width}$ ), baru kemudian melangkah ke posisi kamar di lantai tersebut ( $+ x$ ). Rumus ini memastikan kita mendarat di titik linear yang tepat dalam memori.

- Apa yang terjadi jika kita mengakses indeks `A[100][100]` pada matriks berukuran 100×100?

Jawaban: Akan terjadi *out-of-bounds*. Karena indeks dimulai dari 0, maka batas maksimum yang sah adalah `A[99][99]`. Mengakses indeks 100 berarti kita mencoba memasuki "kamar" yang tidak ada atau milik program lain, yang bisa menyebabkan program *crash* atau *error segmentation fault*.

### 3. Variabel Lokal vs Global (Scope)

- Di dalam fungsi `apply_blur`, terdapat variabel `float sum`. Apakah variabel `sum` ini bisa diakses dari fungsi `main`? Mengapa?

Jawaban: Tidak bisa. Variabel `sum` bersifat lokal, artinya ia hanya "hidup" dan dikenal di dalam lingkup (scope) fungsi `apply_blur`. Fungsi `main` tidak memiliki akses ke "buku catatan" internal milik fungsi `apply_blur`.

- Apa yang terjadi pada variabel `sum` tersebut saat fungsi `apply_blur` selesai dieksekusi (return)?

Jawaban: Saat fungsi selesai, Stack Pointer (SP) akan ditarik kembali ke posisi semula. Hal ini menyebabkan ruang memori yang tadinya dipakai oleh variabel `sum` di dalam *stack* dianggap kosong dan datanya akan ditimpa oleh fungsi lain. Variabel tersebut "hancur" atau terhapus secara otomatis.

### 4. Struktur Kontrol (Looping)

- Jika gambar berukuran  $W \times H$  dan kernel blur berukuran  $K \times K$ , total berapa kali operasi penjumlahan dilakukan?

Jawaban: Total operasi adalah sebanyak  $W \times H \times K \times K$ . Hal ini karena setiap satu piksel (total  $W \times H$  piksel) harus memeriksa seluruh area tetangganya seluas ukuran kernel (total  $K \times K$  tetangga).

- Mengapa terasa lebih "berat" bekerja saat kita memperbesar ukuran jendela (kernel) blur?

Jawaban: Karena jumlah beban perhitungan meningkat secara eksponensial. Jika kernel diperbesar sedikit saja, jumlah tetangga yang harus dihitung oleh setiap piksel meningkat drastis. Hal ini membuat CPU harus bekerja ekstra keras dan memakan waktu lebih lama untuk menyelesaikan satu gambar.

## PENDAHULUAN

Praktikum ini dirancang untuk mendalami dua konsep fundamental dalam ilmu komputer : manipulasi data citra sebagai representasi data multidimensi dan strategi Divide and Conquer untuk optimasi perkalian matriks. Melalui modul ini, mahasiswa akan menganalisis bagaimana pemilihan algoritma memengaruhi performa program, baik dari segi penggunaan memori (Stack vs Heap) maupun kecepatan eksekusi (kompleksitas waktu).

### Modul 1 : Perkalian Matriks Standar ( $O(n^3)$ )

#### 1. Algoritma Tiga Loop Bersarang

Algoritma ini mengikuti kaidah dasar aljabar linier (baris dikali kolom). Setiap elemen pada matriks hasil C merupakan hasil akumulasi perkalian elemen-elemen yang bersesuaian dari baris matriks A dan kolom matriks B.

```
1  for (int i = 0; i < n; i++) {  
2      for (int j = 0; j < n; j++) {  
3          C[i][j] = 0;  
4          for (int k = 0; k < n; k++) {  
5              C[i][j] += A[i][k] * B[k][j];  
6          }  
7      }  
8  }
```

Listing 1: Cuplikan Kode Perkalian Standar

#### 2. Pertanyaan Analisis

- Apa peran variabel **k** pada loop terdalam? Data apa yang diakses dari matriks A dan matriks B pada setiap iterasi **k**?

**Jawaban :** Variabel **k** berfungsi sebagai indeks penelusur (iterator) yang bergerak secara sinkron menyusuri baris pada matriks A dan kolom pada matriks B. Pada setiap iterasi **k**, program mengakses elemen  $A[i][k]$  (elemen ke- **k** pada baris( **i**)) dan  $B[k][j]$  (elemen ke- **k** pada kolom **j**). Hasil perkalian keduanya ditambahkan ke variabel akumulator untuk menentukan nilai akhir pada posisi  $C[i][j]$ . Tanpa variabel **k**, kita tidak bisa melakukan operasi perkalian titik (*dot product*) antar vektor baris dan kolom.

- Kenapa kita harus membebaskan memori menggunakan `free()` setelah program selesai? Apa yang terjadi pada RAM komputer jika kita terus mengalokasikan memori tanpa pernah membebaskannya (Memory Leak)?

**Jawaban :** Penggunaan `free()` adalah bentuk tanggung jawab manajemen memori manual pada bahasa C. Memori yang dialokasikan di Heap melalui `malloc` tidak akan terhapus secara otomatis meskipun fungsi telah selesai. Jika `free()` diabaikan, akan terjadi Memory Leak, di mana RAM tetap "mengunci" ruang memori tersebut meskipun program sudah tidak memakainya. Jika ini terjadi terus-menerus, kapasitas RAM akan habis, menyebabkan sistem melambat, dan akhirnya memaksa sistem operasi menutup program secara paksa (*crash*).

- Bagaimana jumlah operasi perkalian berubah jika ukuran matriks  $n$  menjadi  $2n$ ? Buktikan bahwa kompleksitasnya adalah  $O(n^3)$ .

**Jawaban :** Pada algoritma perkalian matriks standar, digunakan tiga buah loop bersarang yang masing-masing berjalan sebanyak  $n$  kali. Loop pertama digunakan untuk mengakses baris matriks A, loop kedua untuk kolom matriks B, dan loop ketiga untuk melakukan proses perkalian serta penjumlahan elemen. Karena ketiga loop tersebut saling bersarang, maka total operasi yang dilakukan adalah  $n \times n \times n$  atau  $n^3$ . Jika ukuran matriks diperbesar menjadi  $2n$ , maka jumlah operasi menjadi  $(2n)^3 = 8n^3$ , yang menunjukkan bahwa kompleksitas algoritma tetap berada pada orde  $O(n^3)$ .

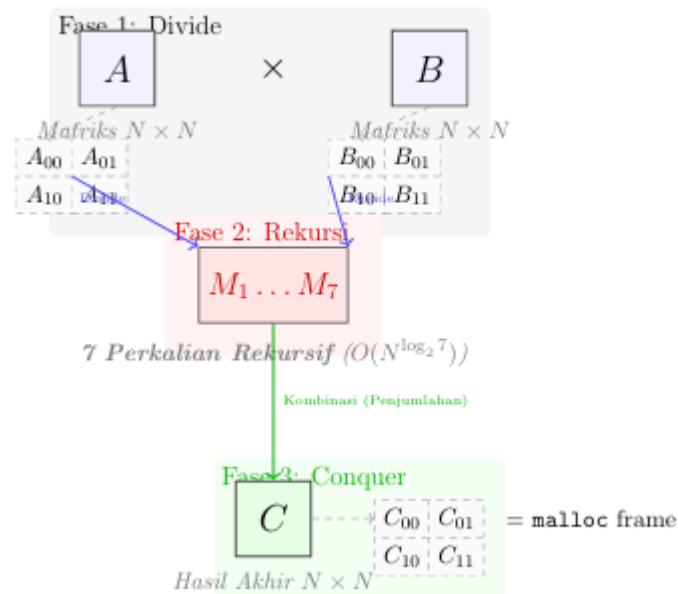
## Modul 2 : Strassen Matrix Multiplication

Deskripsi: Berbeda dengan perkalian matriks standar yang memiliki kompleksitas  $O(n^3)$ , algoritma Strassen mengurangi jumlah perkalian rekursif untuk mengoptimalkan performa pada matriks berukuran besar.

### 1. Visualisasi Strassen: Divide and Conquer

Algoritma Strassen berhasil karena ia memecah matriks besar menjadi sub-matriks yang lebih kecil (Divide) dan menggunakan kombinasi

penjumlahan yang cerdas untuk mengurangi jumlah perkalian rekursif (Conquer).



Gambar 1 : Visualisasi Aliran Divide and Conquer pada Strassen: Matriks A dan B dipecah (Divide) menjadi sub-matriks, diproses secara rekursif menghasilkan 7 nilai M (Rekursi), dan digabungkan kembali (Conquer) menjadi matriks hasil C.

```

1  #include <stdio.h>
2
3  void strassen_2x2(int A[2][2], int B[2][2], int C[2][2]) {
4      // 7 Perkalian Strassen
5      int m1 = (A[0][0] + A[1][1]) * (B[0][0] + B[1][1]);
6      int m2 = (A[1][0] + A[1][1]) * B[0][0];
7      int m3 = A[0][0] * (B[0][1] - B[1][1]);
8      int m4 = A[1][1] * (B[1][0] - B[0][0]);
9      int m5 = (A[0][0] + A[0][1]) * B[1][1];
10     int m6 = (A[1][0] - A[0][0]) * (B[0][0] + B[0][1]);
11     int m7 = (A[0][1] - A[1][1]) * (B[1][0] + B[1][1]);
12
13     // Kombinasi menjadi hasil akhir
14     C[0][0] = m1 + m4 - m5 + m7;
15     C[0][1] = m3 + m5;
16     C[1][0] = m2 + m4;
17     C[1][1] = m1 - m2 + m3 + m6;
18 }
19
20 int main() {
21     int A[2][2] = {{1, 2}, {3, 4}};

```

```

22     int B[2][2] = {{5, 6}, {7, 8}};
23     int C[2][2];
24
25     strassen_2x2(A, B, C);
26
27     printf("Hasil Strassen 2x2:\n");
28     for(int i=0; i<2; i++) {
29         for(int j=0; j<2; j++) printf("%d ", C[i][j]);
30         printf("\n");
31     }
32     return 0;
33 }

```

Listing 2: Program perkalian Matriks dengan optimasi Strassen

### Pertanyaan Analisis

- Apa keuntungan menggunakan malloc dibandingkan array statis pada kasus ini?

Jawaban : Keuntungan **malloc** dibanding Array Statis **malloc** memungkinkan alokasi memori secara dinamis di Heap, sehingga program dapat menangani matriks berukuran sangat besar yang mungkin melampaui kapasitas Stack. Selain itu, ukuran matriks dapat ditentukan saat *runtime* sesuai kebutuhan pengguna.

- Kenapa inisialisasi random penting untuk menguji validitas algoritma pada berbagai distribusi data?

Jawaban : Inisialisasi random memastikan algoritma diuji dengan berbagai variasi angka. Hal ini penting untuk memvalidasi bahwa logika penjumlahan dan perkalian dalam Strassen tetap akurat pada berbagai distribusi data, bukan hanya pada angka-angka sederhana.

- Bagaimana pergerakan Stack Pointer saat fungsi strassen memanggil dirinya sendiri secara rekursif?

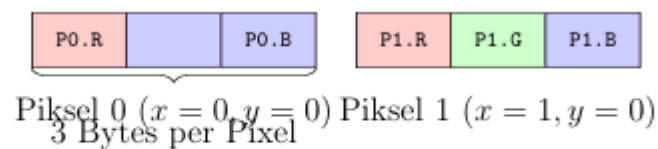
Jawaban : Saat Rekursi Setiap kali fungsi **strassen** memanggil dirinya sendiri, Stack Pointer akan bergerak turun (ke alamat memori yang lebih rendah) untuk mengalokasikan *stack frame* baru bagi fungsi tersebut. *Frame* ini menyimpan parameter, variabel lokal, dan alamat kembali. Saat fungsi selesai (*return*), Stack Pointer akan naik kembali ke posisi sebelumnya.

### Modul 3: Konvolusi 2D (Image Blurring)

Deskripsi: Citra digital direpresentasikan sebagai matriks piksel. Operasi blurring dilakukan dengan merata-ratakan nilai piksel di sekitar tetangganya menggunakan jendela (kernel) berukuran  $K \times K$

#### 1. Struktur Memori: Interleaved RG

Jika gambar memiliki 3 channels (Red, Green, Blue), maka setiap piksel diwakili oleh 3 elemen unsigned char yang letaknya berdampingan. Strukturnya di memori terlihat seperti ini: [R][G][B] [R][G][B] [R][G][B] ... Indeks  $i$  adalah Red. Indeks  $i + 1$  adalah Green. Indeks  $i + 2$  adalah Blue.



$$\text{Index} = (y * \text{width} + x) * \text{channels} + \text{channel\_offset}$$

Gambar 2: Representasi Memori Interleaved RGB: Data R, G, dan B disimpan berurutan untuk setiap piksel.

Input Image ( $5 \times 5$ )

|    |    |    |    |    |
|----|----|----|----|----|
| 10 | 20 | 10 | 0  | 5  |
| 30 | 40 | 30 | 10 | 0  |
| 20 | 10 | 20 | 20 | 10 |
| 40 | 30 | 40 | 30 | 0  |
| 10 | 0  | 10 | 0  | 5  |

Kernel ( $3 \times 3$ )

|       |       |       |
|-------|-------|-------|
| $1/9$ | $1/9$ | $1/9$ |
| $1/9$ | $1/9$ | $1/9$ |
| $1/9$ | $1/9$ | $1/9$ |

Output ( $3 \times 3$ )

|     |     |     |
|-----|-----|-----|
| ... | ... | ... |
| ... | 28  | ... |
| ... | ... | ... |

Kuning: Jendela Geser (Window)

Gambar 3: Ilustrasi Operasi Konvolusi: Setiap elemen di jendela kuning dikalikan dengan elemen kernel yang bersesuaian, lalu dijumlahkan untuk menghasilkan satu nilai di matriks output



## 2. Mengapa Loop Ketiga (for ch ; channels) Diperlukan?

Dalam program blurring yang kita buat, terdapat tiga lapis loop utama:

Loop Y : Menelusuri baris gambar.

Loop X : Menelusuri kolom gambar.

Loop CH : Menelusuri komponen warna (R, G, atau B). Logikanya: Kita tidak bisa merata-ratakan nilai Merah dengan nilai Hijau. Kita harus menghitung rata-rata Merah dari tetangga-tetangganya, lalu rata-rata Hijau dari tetangganya, dan seterusnya. Itulah mengapa perhitungan sum dilakukan di dalam loop ch.

```
1 #define STB_IMAGE_IMPLEMENTATION
2 #include "stb_image.h"
3 #define STB_IMAGE_WRITE_IMPLEMENTATION
4 #include "stb_image_write.h"
5
6 void apply_blur(unsigned char* in, unsigned char* out, int w, int
7 h, int c, int k_size) {
8     int r = k_size / 2;
9     for (int y = 0; y < h; y++) {
10         for (int x = 0; x < w; x++) {
11             for (int ch = 0; ch < c; ch++) {
12                 float sum = 0; int count = 0;
13                 for (int ky = -r; ky <= r; ky++) {
14                     for (int kx = -r; kx <= r; kx++) {
15                         int py = y + ky; int px = x + kx;
16                         if (px >= 0 && px < w && py >= 0 && py <
17                             h) {
18                             sum += in[(py * w + px) * c + ch];
19                             count++;
20                         }
21                     }
22                 }
23                 out[(y * w + x) * c + ch] = (unsigned char)(sum /
24 count);
25             }
26         }
27     }
28 }
```

Listing 3: Konvolusi 2D menggunakan stb\_image

## 3. Pertanyaan Analisis

- Apa yang terjadi jika kita tidak melakukan pengecekan batas (boundary check) pada koordinat piksel?

Jawaban : Jika pengecekan batas tidak dilakukan, program akan mencoba mengakses indeks memori di luar rentang

array yang dialokasikan (misalnya koordinat negatif atau melebihi lebar/tinggi gambar). Hal ini dapat menyebabkan **Segmentation Fault** (*crash* pada program) atau menghasilkan data "sampah" yang merusak tampilan visual citra hasil.

- Kenapa algoritma ini memiliki kompleksitas  $O(N^2 \cdot K^2)$ ?

Jawaban : Kompleksitas ini muncul karena algoritma harus menelusuri setiap piksel pada gambar berukuran  $N \times N$  (total  $N^2$  piksel). Untuk setiap piksel tersebut, program melakukan iterasi tambahan seluas ukuran jendela kernel  $K \times K$  (total  $K^2$  operasi) untuk menghitung nilai rata-rata tetangganya. Karena kedua proses ini bersifat bersarang (*nested*), maka total operasinya adalah perkalian kedua

- Bagaimana cara memodifikasi algoritma ini agar lebih cepat menggunakan teknik Separable Convolution?

Jawaban : Cara memodifikasinya adalah dengan memecah satu proses konvolusi 2D ( $K \times K$ ) menjadi dua tahap konvolusi 1D secara terpisah. Pertama, lakukan konvolusi secara horizontal menggunakan kernel berukuran  $1 \times K$ , kemudian hasilnya diproses kembali secara vertikal menggunakan kernel  $K \times 1$ . Teknik ini menurunkan kompleksitas dari perkalian ( $K \cdot K$ ) menjadi penjumlahan ( $K + K$ ), sehingga proses menjadi jauh lebih cepat.